OPIPE-02: Span Processing & Enrichment

> **Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 2 of 7 | **Created:** March 2026 | **Last Updated:** 05/06/2026

Filtering, Enriching, and Routing Distributed Traces at Ingestion

Span data from distributed tracing can be high-volume and noisy. Health check probes, readiness endpoints, and internal synthetic calls generate spans that consume storage but rarely provide troubleshooting value. OpenPipeline's **Spans scope** lets you filter, enrich, and route trace data before it reaches Grail — reducing cost and improving signal-to-noise ratio.

This notebook covers the span-specific capabilities of OpenPipeline. For general pipeline architecture, see **OPIPE-01: OpenPipeline as a Multi-Scope Platform**. For querying spans after processing, see **SPANS-01: Fundamentals**.

Table of Contents

1. Understanding Span Data in OpenPipeline
2. Span Filtering: Dropping Noise at Ingestion
3. Span Attribute Enrichment
4. Span-to-Log and Span-to-Event Generation
5. Routing Spans to Dedicated Buckets
6. Monitoring Your Span Pipeline
7. Summary
8. Next Steps

9. References

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail and distributed tracing enabled
Permissions	`storage:spans:read`, `openpipeline:configurations:write`
Data	At least 1 hour of span data from instrumented services
Recommended	**OPIPE-01** (multi-scope architecture) and **SPANS-01** (span fundamentals)

1. Understanding Span Data in OpenPipeline

Before configuring span processing, you need to understand what fields are available for routing and filtering decisions.

Key Span Fields for Pipeline Configuration

Field	Description	Pipeline Use
`span.kind`	`server`, `client`, `internal`, `producer`, `consumer`	Route by span type
`service.name`	The service that generated the span	Route by service
`span.name`	The operation name (e.g., `GET /health`)	Filter noise
`http.route`	The HTTP route pattern	Filter health checks
`http.request.method`	HTTP method (GET, POST, etc.)	Filter by method

Field	Description	Pipeline Use
`http.response.status_code`	HTTP response code	Filter or extract metrics
`db.system`	Database type (mysql, postgresql, redis)	Route DB spans
`k8s.namespace.name`	Kubernetes namespace	Route by environment
`dt.entity.service`	Dynatrace service entity ID	Route by service entity

Discovering Your Span Landscape

```
// Span volume by kind and service (top 20)
fetch spans, from:-1h
| summarize span_count = count(), by:{span.kind, service.name}
| sort span_count desc
| limit 20
```

```
// Top span operations by volume (candidates for filtering)
fetch spans, from:-1h
| summarize span_count = count(), by:{span.name, service.name}
| sort span_count desc
| limit 20
```

2. Span Filtering: Dropping Noise at Ingestion

Span filtering is the highest-impact optimization in the spans scope. Typical environments generate 30-60% of span volume from operations that provide no troubleshooting value.

Common Noise Patterns to Drop

Pattern	Filter Condition	Rationale
Health check probes	`span.name` matches `"GET /health*"` or `"GET /ready*"`	Kubernetes liveness/readiness probes
Synthetic monitoring	`span.name` matches `"*synthetic*"`	Internal synthetic test traffic
Internal heartbeats	`span.kind == "internal"` AND `span.name` matches `"*heartbeat*"`	Framework-generated heartbeats
Metrics scraping	`http.route` matches `"/metrics"` or `"/actuator/prometheus"`	Prometheus scrape endpoints
OPTIONS preflight	`http.request.method == "OPTIONS"`	CORS preflight requests

Measuring the Impact

Before configuring drop rules, quantify how much volume each noise pattern represents:

```
// Quantify noise: health checks, synthetic, and OPTIONS requests
fetch spans, from:-1h
| summarize total = count(),
    health_checks = countIf(matchesValue(span.name, "GET /health*") or
matchesValue(span.name, "GET /ready*") or matchesValue(span.name, "GET
/livez*")),
    metrics_scrapes = countIf(matchesValue(span.name, "*/metrics*") or
matchesValue(span.name, "*/actuator/*")),
    options_preflight = countIf(http.request.method == "OPTIONS")
| fieldsAdd noise_total = health_checks + metrics_scrapes + options_preflight
| fieldsAdd noise_pct = round(toDouble(noise_total) / toDouble(total) * 100,
decimals: 1)
```

```
// Detail: Top 10 span names that look like noise (short-lived, high-
frequency)
fetch spans, from:-1h
| summarize span_count = count(), avg_duration = avg(duration), by:{span.name}
| sort span_count desc
| limit 10
| fieldsAdd avg_duration_ms = round(toDouble(avg_duration) / 1000000,
decimals: 2)
```

3. Span Attribute Enrichment

Enrichment adds fields to spans at ingestion that would be expensive or impossible to add at query time. Common enrichment patterns:

Business Context Enrichment

Add business-meaningful attributes based on technical fields:

Condition	Enrichment Field	Value	Purpose
`service.name` starts with `"checkout"`	`business.domain`	`"commerce"`	Business domain tagging
`k8s.namespace.name` contains `"prod"`	`environment`	`"production"`	Environment classification
`http.response.status_code` >= 500	`incident.severity`	`"high"`	Severity pre-classification
`db.system` is not null	`span.category`	`"database"`	Span categorization

Security Context Enrichment

Add `dt.security_context` based on service ownership (see **OPIPE-01** and **ORGNZ-06** for details):

Condition	`dt.security_context` Value
`k8s.namespace.name` matches `"team-alpha-*"`	`"team-alpha"`

Condition	`dt.security_context` Value
`service.name` matches `"payment-*"`	`"payment-team"`
`db.system` is not null	`"database-team"`

Verifying Enrichment

```
// Check which spans have security context enrichment
fetch spans, from:-1h
| summarize total = count(),
    with_security_context = countIf(isNotNull(dt.security_context)),
    by:{service.name}
| fieldsAdd coverage_pct = round(toDouble(with_security_context) /
toDouble(total) * 100, decimals: 1)
| sort total desc
| limit 15
```

4. Span-to-Log and Span-to-Event Generation

OpenPipeline can generate **log records** or **events** from span data at ingestion. This is useful for:

- **Error alerting**: Generate an event when a span has `http.response.status_code >= 500`, enabling Dynatrace Intelligence to detect error spikes without querying spans directly
- **Audit logging**: Create a log record for every span that touches a sensitive service, providing an audit trail in the logs scope
- **SLO tracking**: Generate business events from spans that represent key user transactions

Generation vs. Metric Extraction

Feature	Span-to-Event Generation	Metric Extraction from Spans
Output	Individual event records	Aggregated metric data points
Use case	Alerting, audit trails	Dashboards, SLOs, trend analysis
Volume	1 event per matching span	1 data point per aggregation interval

Feature	Span-to-Event Generation	Metric Extraction from Spans
Cost	Higher (more records)	Lower (aggregated)

Metric extraction from spans is covered in detail in **OPIPE-03: Sampling-Aware Metrics**.

```
// Identify error spans that could trigger event generation
fetch spans, from:-1h
| filter http.response.status_code &gt;= 500
| summarize error_count = count(), by:{service.name,
http.response.status_code}
| sort error_count desc
| limit 15
```

5. Routing Spans to Dedicated Buckets

Just as logs benefit from bucket separation (see **OPLOGS-04: Buckets & Data Governance**), spans benefit from targeted routing.

Span Routing Strategy

Pipeline	Routing Condition	Target Bucket	Retention	Rationale
`frontend-traces`	`span.kind == "server"` AND `service.name` matches frontend	`frontend_spans`	14 days	High volume, short-term debugging
`api-traces`	`span.kind == "server"` AND `http.route` starts with `"/api"`	`api_spans`	35 days	API SLO tracking
`database-traces`	`span.kind == "client"` AND `db.system` is not null	`db_spans`	14 days	Query performance analysis
`messaging-traces`	`span.kind` in `{"producer", "consumer"}`	`messaging_spans`	14 days	Async flow debugging

Pipeline	Routing Condition	Target Bucket	Retention	Rationale
Default Pipeline	Everything else	`default_spans`	7 days	Low-retention catch-all

Verifying Bucket Distribution

```
// Current span bucket distribution
fetch spans, from:-1h
| summarize span_count = count(), by:{dt.system.bucket}
| sort span_count desc
```

```
// Span volume by kind and bucket (is routing working correctly?)
fetch spans, from:-1h
| summarize span_count = count(), by:{span.kind, dt.system.bucket}
| sort span_count desc
```

6. Monitoring Your Span Pipeline

After configuring span processing rules, monitor their effectiveness over time.

```
// Span volume trend by pipeline (are drop rules reducing volume?)
fetch spans, from:-24h
| makeTimeseries span_count = count(), by:{dt.openpipeline.pipelines},
interval:1h
```

```
// Span volume by service over 24h (spot unexpected spikes)
fetch spans, from:-24h
| makeTimeseries span_count = count(), by:{service.name}, interval:1h
```

Summary

In this notebook you learned:

- **Span landscape discovery** — Key fields for routing and filtering decisions

- **Noise filtering** — Dropping health checks, synthetic traffic, and preflight requests at ingestion
- **Attribute enrichment** — Adding business context and security context to spans
- **Event generation** — Creating events and log records from span data for alerting and audit
- **Bucket routing** — Separating spans by purpose with differentiated retention
- **Pipeline monitoring** — Tracking volume trends to verify filtering effectiveness

---

Next Steps

Continue to **OPIPE-03: Sampling-Aware Metrics** to learn how to extract accurate metrics from sampled span data — including the RED metrics (Rate, Errors, Duration) that drive SLOs and dashboards.

References

- OpenPipeline Spans
- Span Processing Rules
- Grail Bucket Management
